

■ Project Files & Folder Structure Explained

Case Study 139: Soil Classification Project

A Complete Guide explaining WHAT, WHY, HOW, and WHEN each file and folder is used in this repository.

■■ Organized Project Architecture Overview

```
/Users/rayhaansiledar/Desktop/ML Final Project/
■■■ app.py # ■ Streamlit Interactive Web Application (Live at port 8503)
■■■ train_and_evaluate.py # ■ Automated Training, Cross-Validation & Plot Pipeline
■■■ Soil_Classification_Case_Study_139.ipynb # ■ Step-by-step Executed Jupyter Notebook
■■■ requirements.txt # ■ Python Dependencies List
■■■ README.md # ■ Main GitHub Project Documentation
■
■■■ docs/ # ■ All Documentation & PDFs
■ ■■■ markdown/ # ■ Markdown source files
■ ■ ■■■ PROJECT_REPORT.md # Formal 16-Chapter Academic Report
■ ■ ■■■ VIVA_PREPARATION_GUIDE.md # 25+ High-Yield Viva Voce Q&As
■ ■ ■■■ ML_TOPICS_EXPLAINED.md # ML Concepts Guide (HOW, WHY, WHEN)
■ ■ ■■■ PROJECT_FILES_EXPLAINED.md # This File (File-by-File Breakdown)
■ ■ ■■■ ML_MODELS_EXPLAINED.md # 5 ML Models Deep Dive Guide
■ ■■■ pdf/ # ■ Exported Publication-Grade PDFs
■ ■■■ PROJECT_REPORT.pdf
■ ■■■ VIVA_PREPARATION_GUIDE.pdf
■ ■■■ ML_TOPICS_EXPLAINED.pdf
■ ■■■ PROJECT_FILES_EXPLAINED.pdf
■ ■■■ ML_MODELS_EXPLAINED.pdf
■ ■■■ README.pdf
■
■■■ dataset/ # ■ Raw & Processed Soil Data
■ ■■■ soil_classification.csv # Main 2,800-sample balanced dataset
■ ■■■ online_raw_soil_measures.csv # Live downloaded online agricultural dataset
■ ■■■ test_samples.csv # 25-sample batch test template
■
■■■ models/ # ■ Serialized Models & Benchmarks
■ ■■■ logistic_regression.pkl # Trained Logistic Regression model
■ ■■■ knn.pkl # Trained KNN model
■ ■■■ decision_tree.pkl # Trained Decision Tree model
■ ■■■ random_forest.pkl # Trained Random Forest model
■ ■■■ gradient_boosting.pkl # Trained Gradient Boosting model
■ ■■■ scaler.pkl # Trained StandardScaler object
■ ■■■ label_encoder.pkl # Trained LabelEncoder object
■ ■■■ benchmark_comparison.csv # Comparative Leaderboard table
■ ■■■ feature_importance.csv # Gini & Boosting feature ranking
■ ■■■ preprocessing_impact.csv # Raw vs. Scaled performance comparison
■ ■■■ confused_soil_pairs.csv # Misclassified class error pairs
■ ■■■ model_metadata.json # Full project JSON metadata
■
■■■ eda_plots/ # ■ 7 High-Resolution EDA Charts (PNG)
■■■ model_metrics/ # ■ 6 Model Evaluation Charts (PNG)
■■■ scripts/ # ■ ■ Helper & Utility Scripts
■■■ convert_docs_to_pdf.py # Markdown-to-PDF batch converter script
```

1. _CODE_PH_0_ (Streamlit Interactive Web Application)

■ What is it?

app.py is the production web application of the project. It builds a modern, glassmorphic interactive graphical user interface (GUI) using the Python `_CODE_PH_1_` framework.

■ WHY do we need it?

- A machine learning model saved in a Python script is invisible to normal users.
- **app.py** allows farmers, agronomists, examiners, and students to interact with all 5 trained models in real time through web sliders, upload CSV files for batch testing, and explore charts without writing code.

■■ HOW does it work?

1. **Resource Loading:** Uses `_CODE_PH_0_` to load the serialized models (`.pkl`) and scaler into memory in milliseconds.
2. **Interactive Controls:** Provides input sliders for **pH, Nitrogen, Phosphorus, Potassium, Moisture, Organic Matter, Electrical Conductivity, Temperature, and Humidity**.
3. **Multi-Model Inference:** Lets the user toggle between all 5 ML models (Logistic Regression, KNN, Decision Tree, Random Forest, Gradient Boosting) and computes instant predictions.
4. **5 Tabbed Modules:**
 - **Tab 1 (Live Soil Classifier):** Real-time prediction card, confidence gauge, probability distribution bar chart, and tailored crop/fertilizer recommendations.
 - **Tab 2 (Batch CSV Assessment):** Allows users to upload a spreadsheet of soil tests, runs batch predictions, and offers a downloadable classified CSV report.
 - **Tab 3 (Model Benchmark & Comparison):** Interactive multi-metric leaderboard and interactive confusion matrix heatmap viewer.
 - **Tab 4 (Exploratory Data Analysis):** Interactive Plotly visualizations (boxplots, scatter plots, 3D nutrient charts).
 - **Tab 5 (Case Study & Viva Hub):** Direct answers to all 6 Case Study 139 questions and viva prep accordion.

■■ WHEN is it executed?

- Run when you want to launch the live dashboard:

```
streamlit run app.py --server.port 8503
```

2. `_CODE_PH_0_` (Training & Evaluation Pipeline)

■ What is it?

train_and_evaluate.py is the reproducible master Python script that runs the entire end-to-end data science pipeline from scratch.

■ WHY do we need it?

To ensure complete scientific reproducibility. Anyone can run this single command to re-download the data, train all 5 models, run 5-Fold Stratified Cross Validation, calculate all metrics, generate charts, and save `_CODE_PH_0_` files.

■■ HOW does it work?

1. **Data Ingestion (fetch_online_soil_dataset):** Downloads open online agricultural data from GitHub, generates 2,800 balanced observations across 8 soil classes, and saves `_CODE_PH_1_`.

- 2. EDA (generate_eda_visualizations):** Generates and saves 7 publication-grade PNG charts to `_CODE_PH_1_`.
- 3. Preprocessing Impact Study (evaluate_preprocessing_impact):** Compares all 5 models on unscaled vs. `_CODE_PH_1_` data, saving `_CODE_PH_2_`.
- 4. Model Training & Cross-Validation:** Trains Logistic Regression, KNN, Decision Tree, Random Forest, and Gradient Boosting under 5-Fold Stratified CV.
- 5. Serialization:** Exports all 5 models, the scaler, label encoder, metadata JSON, and benchmark CSVs into the `_CODE_PH_0_` folder.
- 6. Chart Generation:** Creates confusion matrices and accuracy comparison bar charts in `_CODE_PH_0_`.

■ ■ WHEN is it executed?

- Run whenever you want to re-train the models or re-generate metrics:

```
python3 train_and_evaluate.py
```

3. `_CODE_PH_0_` (Jupyter Notebook)

■ What is it?

An interactive, fully executed Jupyter Notebook containing every step of the project with markdown explanations, runnable Python code cells, inline graphs, and scientific takeaways.

■ WHY do we need it?

- Standard submission format for academic coursework and university evaluations.
- Allows professors and examiners to inspect intermediate outputs, variable states, and inline graphs cell-by-cell.

■ ■ WHEN is it used?

- Opened in JupyterLab or VS Code during viva presentations or academic code reviews:

```
jupyter notebook Soil_Classification_Case_Study_139.ipynb
```

4. `_CODE_PH_0_` (Academic Project Report)

■ What is it?

A comprehensive, formal 16-chapter academic project report structured specifically for 100-mark university project evaluations.

■ WHY do we need it?

Provides complete written documentation containing:

- Abstract & Problem Formulation
- Literature Survey & Agronomic Fundamentals
- Geochemical characterization of all 8 soil types
- Detailed mathematical formulations of all 5 algorithms
- Experimental tables and 5-Fold Cross-Validation leaderboards
- Granular confusion matrix error analysis

- Definitive answers to all 6 case study research questions
 - References and academic citations
-

5. `_CODE_PH_0_` (Viva Voce Cheat Sheet)

■ What is it?

A focused preparation guide featuring 25+ curated viva questions and model answers designed to help students score 100% in oral examinations.

■ WHY do we need it?

Covers examiner trap questions, mathematical derivations (Gini, Sigmoid, Euclidean distance, boosting residuals), cross-validation nuances, and agronomic justifications.

6. `_CODE_PH_0_` (Dependencies File)

■ What is it?

A text file listing all external Python packages required to run the project with version constraints:

- **streamlit** (Web dashboard framework)
- **scikit-learn** (Machine learning algorithms and metrics)
- **pandas** (Dataframe manipulation)
- **numpy** (Numerical array computing)
- **matplotlib** & `_CODE_PH_1_` (Static publication charts)
- **plotly** (Interactive browser charts)
- **joblib** (Model serialization)
- **statsmodels** (Statistical trend analysis)

■ WHEN is it used?

- Run during initial environment setup:

```
python3 -m pip install -r requirements.txt
```

7. `_CODE_PH_0_` Directory

■ What is inside?

1. **soil_classification.csv**: The primary dataset containing 2,800 balanced soil observations across 8 distinct soil categories with 9 physical/chemical attributes.
 2. **online_raw_soil_measures.csv**: Real-world raw agricultural data downloaded live from GitHub.
 3. **test_samples.csv**: A 25-sample unlabeled test template for testing the Streamlit app's batch CSV upload feature.
-

8. `_CODE_PH_0_` Directory

■ What is inside?

1. `logistic_regression.pkl`, `_CODE_PH_1_`, `_CODE_PH_2_`, `_CODE_PH_3_`, `_CODE_PH_4_`: The 5 trained models serialized via `_CODE_PH_5_`.
 2. `scaler.pkl`: The fitted `_CODE_PH_1_` preserving training feature means (μ) and standard deviations (σ).
 3. `label_encoder.pkl`: The fitted `_CODE_PH_1_` mapping soil names to integer IDs.
 4. `benchmark_comparison.csv`: Quantitative metrics table (Accuracy, Precision, Recall, Macro F1, Weighted F1, 5-Fold CV).
 5. `feature_importance.csv`: Relative feature weights from Random Forest and Gradient Boosting.
 6. `preprocessing_impact.csv`: Raw vs. scaled accuracy comparison data.
 7. `confused_soil_pairs.csv`: Table of misclassified soil pairs for error analysis.
 8. `model_metadata.json`: Structured JSON containing all scores, parameters, and metadata for fast programmatic loading.
-

9. `_CODE_PH_0_` and `_CODE_PH_1_` Directories

■ What is inside?

High-resolution 300 DPI PNG visual assets generated by the training pipeline:

- `01_soil_type_distribution.png`
- `02_correlation_matrix.png`
- `03_soil_properties_boxplots.png`
- `04_npk_ratio_distribution.png`
- `05_ph_vs_ec_scatter.png`
- `06_organic_matter_vs_moisture.png`
- `07_radar_soil_profiles.png`
- `01_algorithm_accuracy_comparison.png`
- `02_f1_score_comparison.png`
- `03_multi_metric_radar.png`
- `04_confusion_matrices_grid.png`
- `05_feature_importance_rf_gb.png`
- `06_preprocessing_impact.png`